

flappybird_rl

March 23, 2026

1 Flappy Bird con Deep Reinforcement Learning

1.1 Objetivos de este Notebook

En este notebook aprenderás:

1. **Timing crítico en RL:** Por qué Flappy Bird es un desafío especial
 2. **Observación Simple vs RGB:** Trade-offs de diferentes representaciones
 3. **DQN vs PPO:** Cuándo usar cada algoritmo
 4. **Variantes de arquitectura:** Dueling DQN, Frame Stacking
-

1.2 Prerequisitos

`pip install flappy-bird-gymnasium stable-baselines3`

```
[1]: # Imports necesarios
import sys
import os
from pathlib import Path

# Añadir directorio al path
FLAPPY_DIR = Path().absolute()
if str(FLAPPY_DIR) not in sys.path:
    sys.path.insert(0, str(FLAPPY_DIR))

import numpy as np
import matplotlib.pyplot as plt
from collections import deque
import random

# Gymnasium
import gymnasium as gym
try:
    import flappy_bird_gymnasium
    FLAPPY_AVAILABLE = True
    print("Flappy Bird Gymnasium disponible")
except ImportError:
    FLAPPY_AVAILABLE = False
```

```

print("Instalar con: pip install flappy-bird-gymnasium")

# Stable-Baselines3
from stable_baselines3 import DQN, PPO
from stable_baselines3.common.callbacks import BaseCallback
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.monitor import Monitor
from stable_baselines3.common.vec_env import DummyVecEnv, VecFrameStack

# PyTorch (para variantes custom)
import torch
import torch.nn as nn
import torch.optim as optim

print(f"PyTorch: {torch.__version__}")
print(f"Directorio: {FLAPPY_DIR}")

```

Instalar con: pip install flappy-bird-gymnasium
 PyTorch: 2.10.0+cu128
 Directorio: /home/jeironpro/OPT-ML/Reinforcement
 Learning/03_proyectos_dqn/flappybird

2 1. Descripción del Juego

2.1 ¿Qué es Flappy Bird?

Un juego donde controlas un pájaro que debe pasar entre tubos:

✦ pájaro

2.1.1 ¿Por qué es difícil para RL?

1. **Timing crítico:** El momento exacto de saltar importa mucho
2. **Recompensa sparse:** Solo +1 al pasar un tubo, 0 en otros pasos
3. **Muerte instantánea:** Un error = game over
4. **Física simple pero precisa:** La gravedad y el salto son predecibles pero requieren precisión

```

[2]: # Explorar el entorno
if FLAPPY_AVAILABLE:
    env = gym.make("FlappyBird-v0", use_lidar=True)

```

```

print("="*60)
print("ENTORNO: Flappy Bird")
print("="*60)
print(f"\nEspacio de observación: {env.observation_space}")
print(f"Espacio de acciones: {env.action_space}")

# Ver una observación
obs, info = env.reset()
print(f"\nObservación (12D LiDAR):")
print(f"  Shape: {obs.shape}")
print(f"  Valores: {obs}")

env.close()
else:
    print("Flappy Bird no disponible")

```

Flappy Bird no disponible

3 2. Análisis de la Arquitectura

3.1 2.1 Estado / Observación

3.1.1 Opción A: Observación Simple (12D LiDAR)

Cuando `use_lidar=True`, el entorno devuelve un vector de 12 valores:

Índice	Descripción
0-3	Raycast horizontal (distancia a tubos)
4-7	Raycast vertical (gap de tubos)
8-9	Posición y velocidad del pájaro
10-11	Información del siguiente tubo

3.1.2 Opción B: Observación RGB (288×512×3)

Cuando `use_lidar=False`, el entorno devuelve la imagen completa del juego.

3.1.3 Comparación

Aspecto	Simple (12D)	RGB (288×512)
Velocidad	Muy rápida	Lenta (CNN)
Información	Suficiente	Completa
Red necesaria	MLP simple	CNN profunda
Memoria	~1 KB	~430 KB por frame

```

[3]: # Comparar observaciones
if FLAPPY_AVAILABLE:
    print("="*60)
    print("COMPARACIÓN DE OBSERVACIONES")
    print("="*60)

    # Simple
    env_simple = gym.make("FlappyBird-v0", use_lidar=True)
    obs_simple, _ = env_simple.reset()
    print(f"\nObservación SIMPLE (LiDAR):")
    print(f"  Shape: {obs_simple.shape}")
    print(f"  Tipo: {obs_simple.dtype}")
    print(f"  Memoria: {obs_simple.nbytes} bytes")
    env_simple.close()

    # RGB
    env_rgb = gym.make("FlappyBird-v0", use_lidar=False)
    obs_rgb, _ = env_rgb.reset()
    print(f"\nObservación RGB:")
    print(f"  Shape: {obs_rgb.shape}")
    print(f"  Tipo: {obs_rgb.dtype}")
    print(f"  Memoria: {obs_rgb.nbytes:,} bytes ({obs_rgb.nbytes/1024:.1f} KB)")
    env_rgb.close()

    # Visualizar
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    # Simple como barras
    axes[0].barh(range(12), obs_simple)
    axes[0].set_yticks(range(12))
    axes[0].set_yticklabels([f'Dim {i}' for i in range(12)])
    axes[0].set_xlabel('Valor')
    axes[0].set_title('Observación Simple (12D)')
    axes[0].grid(True, alpha=0.3)

    # RGB como imagen
    axes[1].imshow(obs_rgb)
    axes[1].set_title('Observación RGB')
    axes[1].axis('off')

    plt.tight_layout()
    plt.show()

```

3.2 2.2 Espacio de Acciones

Solo 2 acciones:

Acción	Efecto
0	No hacer nada (caer por gravedad)
1	Saltar (impulso hacia arriba)

3.2.1 ¿Por qué es difícil con solo 2 acciones?

El **timing** es todo. Saltar demasiado pronto o tarde = muerte.

3.3 2.3 Función de Recompensa

Evento	Recompensa
Pasar un tubo	+1.0
Cada paso	+0.1 (pequeño bonus por sobrevivir)
Morir	-1.0 (o -5.0 según configuración)

3.3.1 Problema: Recompensa Sparse

El agente puede pasar muchos pasos sin recompensa positiva significativa.

3.4 2.4 ¿DQN o PPO?

3.4.1 Pregunta: ¿Qué algoritmo funciona mejor para Flappy Bird?

Aspecto	DQN	PPO
Tipo	Value-based	Policy Gradient
Acciones	Solo discretas	Discretas y continuas
Sample efficiency	Mejor (replay buffer)	Peor (on-policy)
Estabilidad	Menos estable	Muy estable
Timing crítico	Regular	Mejor

Conclusión: PPO suele funcionar mejor para juegos con timing crítico porque aprende una **política suave** en lugar de valores Q discretos.

4 3. Código Original

Importamos las funciones del archivo original:

```
[4]: # Callback para registrar scores
class FlappyCallback(BaseCallback):
    """Callback para registrar scores en Flappy Bird."""

    def __init__(self, verbose=0):
        super().__init__(verbose)
```

```

self.scores = []
self.episode_rewards = []
self.best_score = 0

def _on_step(self) -> bool:
    for info in self.locals.get('infos', []):
        if 'episode' in info:
            reward = info['episode']['r']
            self.episode_rewards.append(reward)
            score = max(0, int(reward))
            self.scores.append(score)
            if score > self.best_score:
                self.best_score = score
            if self.verbose > 0:
                print(f" Nuevo récord: {score} tubos!")
    return True

def entrenar_flappy(timesteps=50000, algorithm="DQN", use_simple_obs=True):
    """Entrena un agente en Flappy Bird."""
    print(f"\nEntrenando {algorithm} ({timesteps:,} timesteps)...")

    env = gym.make("FlappyBird-v0", render_mode=None, use_lidar=use_simple_obs)
    env = Monitor(env)

    policy = "MlpPolicy" if use_simple_obs else "CnnPolicy"
    policy_kwargs = {"net_arch": [256, 256]} if use_simple_obs else None

    if algorithm == "DQN":
        model = DQN(
            policy, env, policy_kwargs=policy_kwargs, verbose=0,
            learning_rate=0.0001, buffer_size=50000,
            learning_starts=5000, batch_size=32,
            gamma=0.99, exploration_fraction=0.1,
            exploration_final_eps=0.01, target_update_interval=1000
        )
    else:
        model = PPO(
            policy, env, policy_kwargs=policy_kwargs, verbose=0,
            learning_rate=0.0003, n_steps=2048,
            batch_size=64, n_epochs=10, gamma=0.99
        )

    callback = FlappyCallback(verbose=0)
    model.learn(total_timesteps=timesteps, callback=callback, progress_bar=True)

    env.close()

```

```

    print(f"  Mejor score: {callback.best_score} tubos")
    return model, callback

print("Funciones de entrenamiento cargadas")

```

Funciones de entrenamiento cargadas

5 4. Entrenamiento Rápido (Demo)

Entrenamos brevemente para ver el proceso:

```

[5]: if FLAPPY_AVAILABLE:
    print("="*60)
    print("ENTRENAMIENTO DEMO")
    print("="*60)

    # Entrenar DQN
    model_dqn, cb_dqn = entrenar_flappy(timesteps=30000, algorithm="DQN")
else:
    print("Flappy Bird no disponible")

```

Flappy Bird no disponible

```

[6]: # Visualizar progreso
if FLAPPY_AVAILABLE and cb_dqn.scores:
    plt.figure(figsize=(10, 4))

    scores = cb_dqn.scores
    plt.plot(scores, alpha=0.3, label='Score por episodio')

    if len(scores) > 20:
        smoothed = np.convolve(scores, np.ones(20)/20, mode='valid')
        plt.plot(range(19, len(scores)), smoothed, 'r-', linewidth=2,
        ↪label='Media móvil (20)')

    plt.axhline(y=cb_dqn.best_score, color='green', linestyle='--',
    ↪label=f'Mejor: {cb_dqn.best_score}')
    plt.xlabel('Episodio')
    plt.ylabel('Tubos pasados')
    plt.title('Progreso del Entrenamiento (DQN)')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

```

6 5. VARIANTES DE ARQUITECTURA

6.1 Variante A: DQN vs PPO - Comparación Completa

```
[7]: def comparar_dqn_ppo(timesteps=50000):  
    """  
    Compara DQN vs PPO en Flappy Bird.  
    """  
  
    print("="*60)  
    print("VARIANTE A: DQN vs PPO")  
    print("="*60)  
  
    resultados = {}  
  
    # DQN  
    model_dqn, cb_dqn = entrenar_flappy(timesteps, "DQN")  
    resultados['DQN'] = {'scores': cb_dqn.scores, 'best': cb_dqn.best_score}  
  
    # PPO  
    model_ppo, cb_ppo = entrenar_flappy(timesteps, "PPO")  
    resultados['PPO'] = {'scores': cb_ppo.scores, 'best': cb_ppo.best_score}  
  
    # Visualizar  
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))  
  
    # Curvas  
    for name, data in resultados.items():  
        scores = data['scores']  
        if len(scores) > 20:  
            smoothed = np.convolve(scores, np.ones(20)/20, mode='valid')  
            axes[0].plot(smoothed, label=f"{name} (mejor: {data['best']})")  
  
    axes[0].set_xlabel('Episodio')  
    axes[0].set_ylabel('Tubos pasados')  
    axes[0].set_title('Curvas de Aprendizaje')  
    axes[0].legend()  
    axes[0].grid(True, alpha=0.3)  
  
    # Barras finales  
    names = list(resultados.keys())  
    bests = [resultados[n]['best'] for n in names]  
    axes[1].bar(names, bests, color=['blue', 'orange'])  
    axes[1].set_ylabel('Mejor Score')  
    axes[1].set_title('Mejor Score por Algoritmo')  
  
    plt.tight_layout()  
    plt.show()
```



```

    return resultados

if FLAPPY_AVAILABLE:
    resultados_a = comparar_dqn_ppo(timesteps=30000)

```

6.2 Variante B: Dueling DQN

6.2.1 Idea

Separar la red en dos streams: - **Value stream** $V(s)$: Valor del estado - **Advantage stream** $A(s,a)$: Ventaja de cada acción

$$Q(s,a) = V(s) + A(s,a) - \text{mean}(A)$$

6.2.2 Ventajas

- Aprende qué estados son valiosos independientemente de la acción
- Mejor para juegos donde a veces la acción no importa mucho

```

[8]: class DuelingDQN(nn.Module):
    """
    Dueling DQN: Separa  $V(s)$  y  $A(s,a)$ .

     $Q(s,a) = V(s) + (A(s,a) - \text{mean}(A(s,:)))$ 
    """

    def __init__(self, input_size=12, n_actions=2):
        super().__init__()

        # Encoder compartido
        self.features = nn.Sequential(
            nn.Linear(input_size, 256),
            nn.ReLU(),
            nn.Linear(256, 256),
            nn.ReLU()
        )

        # Value stream:  $V(s)$ 
        self.value_stream = nn.Sequential(
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 1) # Un solo valor
        )

        # Advantage stream:  $A(s,a)$ 
        self.advantage_stream = nn.Sequential(
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, n_actions) # Una ventaja por acción

```

```

    )

    def forward(self, x):
        features = self.features(x)

        value = self.value_stream(features)          # (batch, 1)
        advantage = self.advantage_stream(features)  # (batch, n_actions)

        # Combinar:  $Q = V + (A - \text{mean}(A))$ 
        q_values = value + (advantage - advantage.mean(dim=1, keepdim=True))
        return q_values

class DuelingDQNAgent:
    """Agente con Dueling DQN."""

    def __init__(self, input_size=12, n_actions=2):
        self.n_actions = n_actions
        self.gamma = 0.99
        self.epsilon = 1.0
        self.epsilon_min = 0.01
        self.epsilon_decay = 0.995

        self.device = torch.device("cuda" if torch.cuda.is_available() else ↪
"cpu")

        self.q_network = DuelingDQN(input_size, n_actions).to(self.device)
        self.target_network = DuelingDQN(input_size, n_actions).to(self.device)
        self.target_network.load_state_dict(self.q_network.state_dict())

        self.optimizer = optim.Adam(self.q_network.parameters(), lr=0.0001)
        self.memory = deque(maxlen=50000)
        self.batch_size = 32

    def act(self, state):
        if random.random() < self.epsilon:
            return random.randint(0, self.n_actions - 1)

        with torch.no_grad():
            state_t = torch.FloatTensor(state).unsqueeze(0).to(self.device)
            return self.q_network(state_t).argmax().item()

    def remember(self, state, action, reward, next_state, done):
        self.memory.append((state, action, reward, next_state, done))

    def replay(self):
        if len(self.memory) < self.batch_size:

```

```

        return 0

    batch = random.sample(self.memory, self.batch_size)
    states, actions, rewards, next_states, dones = zip(*batch)

    states_t = torch.FloatTensor(np.array(states)).to(self.device)
    actions_t = torch.LongTensor(actions).to(self.device)
    rewards_t = torch.FloatTensor(rewards).to(self.device)
    next_states_t = torch.FloatTensor(np.array(next_states)).to(self.device)
    dones_t = torch.FloatTensor(dones).to(self.device)

    q_values = self.q_network(states_t).gather(1, actions_t.unsqueeze(1)).
    ↪squeeze()

    with torch.no_grad():
        next_q = self.target_network(next_states_t).max(1)[0]
        target = rewards_t + (1 - dones_t) * self.gamma * next_q

    loss = nn.MSELoss()(q_values, target)

    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    return loss.item()

    def update_target(self):
        self.target_network.load_state_dict(self.q_network.state_dict())

    def decay_epsilon(self):
        self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)

# Verificar arquitectura
dueling_net = DuelingDQN(input_size=12, n_actions=2)
n_params = sum(p.numel() for p in dueling_net.parameters())

print("="*60)
print("VARIANTE B: Dueling DQN")
print("="*60)
print(f"\n{dueling_net}")
print(f"\nParámetros totales: {n_params:,}")

```

```

=====
VARIANTE B: Dueling DQN
=====

```

```

DuelingDQN(

```

```

(features): Sequential(
  (0): Linear(in_features=12, out_features=256, bias=True)
  (1): ReLU()
  (2): Linear(in_features=256, out_features=256, bias=True)
  (3): ReLU()
)
(value_stream): Sequential(
  (0): Linear(in_features=256, out_features=128, bias=True)
  (1): ReLU()
  (2): Linear(in_features=128, out_features=1, bias=True)
)
(advantage_stream): Sequential(
  (0): Linear(in_features=256, out_features=128, bias=True)
  (1): ReLU()
  (2): Linear(in_features=128, out_features=2, bias=True)
)
)

```

Parámetros totales: 135,299

```

[9]: def entrenar_dueling_dqn(epochs=200):
    """Entrena Dueling DQN en Flappy Bird."""
    if not FLAPPY_AVAILABLE:
        print("Flappy Bird no disponible")
        return None, []

    print("\nEntrenando Dueling DQN...")

    env = gym.make("FlappyBird-v0", render_mode=None, use_lidar=True)
    agent = DuelingDQNAgent(input_size=12, n_actions=2)

    scores = []

    for ep in range(epochs):
        state, _ = env.reset()
        total_reward = 0
        done = False

        while not done:
            action = agent.act(state)
            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            agent.remember(state, action, reward, next_state, done)
            agent.replay()

            state = next_state

```

```

        total_reward += reward

    agent.decay_epsilon()
    if ep % 10 == 0:
        agent.update_target()

    score = max(0, int(total_reward))
    scores.append(score)

    if (ep + 1) % 50 == 0:
        print(f"  Ep {ep+1}: Score promedio = {np.mean(scores[-50:]):.1f}")

env.close()
print(f"  Mejor score: {max(scores)}")
return agent, scores

if FLAPPY_AVAILABLE:
    agent_dueling, scores_dueling = entrenar_dueling_dqn(episodes=100)

```

6.3 Variante C: Frame Stacking

6.3.1 Idea

Apilar varios frames consecutivos para dar información temporal: - El agente ve los últimos N estados - Puede inferir velocidad y dirección

6.3.2 Ventajas

- Información temporal sin LSTM/RNN
- Puede mejorar el timing

```

[10]: def entrenar_con_frame_stacking(timesteps=50000, n_stack=4):
    """
    Entrena DQN con frame stacking.

    Args:
        timesteps: Pasos de entrenamiento
        n_stack: Número de frames a apilar
    """
    if not FLAPPY_AVAILABLE:
        print("Flappy Bird no disponible")
        return None, None

    print(f"\nEntrenando DQN con Frame Stacking (n={n_stack})...")

    # Crear entorno con frame stacking
    def make_env():
        env = gym.make("FlappyBird-v0", render_mode=None, use_lidar=True)

```

```

    return Monitor(env)

env = DummyVecEnv([make_env])
env = VecFrameStack(env, n_stack=n_stack)

# El estado ahora tiene shape (12 * n_stack,) = (48,) para n_stack=4
print(f" Observación original: 12D")
print(f" Observación con stacking: {12 * n_stack}D")

model = DQN(
    "MlpPolicy", env,
    policy_kwargs={"net_arch": [256, 256]},
    verbose=0,
    learning_rate=0.0001,
    buffer_size=50000,
    learning_starts=5000,
    batch_size=32,
    gamma=0.99
)

callback = FlappyCallback(verbose=0)
model.learn(total_timesteps=timesteps, callback=callback, progress_bar=True)

print(f" Mejor score: {callback.best_score}")
return model, callback

if FLAPPY_AVAILABLE:
    model_stacked, cb_stacked = entrenar_con_frame_stacking(timesteps=30000,
↪n_stack=4)

```

6.4 Variante D: Simple vs RGB - Comparación Detallada

```

[11]: def comparar_observaciones(timesteps=30000):
    """
    Compara observación simple (12D) vs RGB.
    """
    if not FLAPPY_AVAILABLE:
        print("Flappy Bird no disponible")
        return {}

    print("="*60)
    print("VARIANTE D: Simple vs RGB")
    print("="*60)

    import time
    resultados = {}

```

```

# Simple
print("\nEntrenando con observación SIMPLE (12D)...")
t0 = time.time()
model_simple, cb_simple = entrenar_flappy(timesteps, "DQN",
↪use_simple_obs=True)
time_simple = time.time() - t0
resultados['Simple (12D)'] = {
    'scores': cb_simple.scores,
    'best': cb_simple.best_score,
    'time': time_simple
}

# RGB (más lento)
print("\nEntrenando con observación RGB (esto tardará más)...")
t0 = time.time()
model_rgb, cb_rgb = entrenar_flappy(timesteps // 2, "DQN",
↪use_simple_obs=False) # Menos timesteps porque es lento
time_rgb = time.time() - t0
resultados['RGB'] = {
    'scores': cb_rgb.scores,
    'best': cb_rgb.best_score,
    'time': time_rgb
}

# Tabla comparativa
print("\n" + "="*60)
print("RESULTADOS")
print("="*60)
print(f"\n{'Observación':<15} {'Mejor Score':<15} {'Tiempo (s)':<15}")
print("-" * 45)
for name, data in resultados.items():
    print(f"{name:<15} {data['best']:<15} {data['time']:<15.1f}")

return resultados

if FLAPPY_AVAILABLE:
    resultados_d = comparar_observaciones(timesteps=20000)

```

7 6. Comparación Final de Todas las Variantes

```

[12]: # Recopilar todos los resultados
if FLAPPY_AVAILABLE:
    print("="*60)
    print("RESUMEN DE TODAS LAS VARIANTES")
    print("="*60)

```

```

variantes = [
    ("DQN Original", cb_dqn.scores if 'cb_dqn' in dir() else []),
    ("Dueling DQN", scores_dueling if 'scores_dueling' in dir() else []),
    ("Frame Stacking", cb_stacked.scores if 'cb_stacked' in dir() and
↪cb_stacked else []),
]

# Filtrar variantes con datos
variantes = [(n, s) for n, s in variantes if len(s) > 0]

if variantes:
    plt.figure(figsize=(12, 5))

    # Curvas
    plt.subplot(1, 2, 1)
    for name, scores in variantes:
        if len(scores) > 20:
            smoothed = np.convolve(scores, np.ones(20)/20, mode='valid')
            plt.plot(smoothed, label=f"{name} (mejor: {max(scores)})")
    plt.xlabel('Episodio')
    plt.ylabel('Score')
    plt.title('Comparación de Variantes')
    plt.legend()
    plt.grid(True, alpha=0.3)

    # Barras
    plt.subplot(1, 2, 2)
    names = [n for n, _ in variantes]
    bests = [max(s) for _, s in variantes]
    plt.bar(names, bests)
    plt.ylabel('Mejor Score')
    plt.title('Mejor Score por Variante')
    plt.xticks(rotation=15)

    plt.tight_layout()
    plt.show()

    # Tabla
    print(f"\n{'Variante':<20} {'Mejor':<10} {'Promedio':<10}")
    print("-" * 40)
    for name, scores in variantes:
        print(f"{name:<20} {max(scores):<10} {np.mean(scores[-20:]):<10.
↪1f}")

```


8 7. Conclusiones

8.1 ¿Qué aprendimos?

1. **Observación Simple vs RGB:**
 - La observación simple (12D LiDAR) es suficiente y mucho más rápida
 - RGB es overkill para este juego simple
2. **DQN vs PPO:**
 - PPO suele ser más estable para juegos con timing crítico
 - DQN puede requerir más tuning
3. **Variantes de arquitectura:**
 - Dueling DQN puede ayudar cuando el valor del estado es importante
 - Frame stacking añade información temporal

8.2 Sigüientes Pasos

- Entrenar por más timesteps (500K+)
- Probar Prioritized Experience Replay
- Experimentar con diferentes hiperparámetros
- Añadir reward shaping

8.3 Referencias

- [Dueling Network Architectures for Deep RL](#)
- [PPO: Proximal Policy Optimization](#)
- [Stable-Baselines3 Documentation](#)

8.4 Variantes de Entrenamiento — Flappy Bird

Las variantes en Flappy Bird exploran dos dimensiones independientes: - **Algoritmo:** DQN (off-policy) vs PPO (on-policy) - **Observación:** Simple 12D (LIDAR) vs RGB completo (imagen)

Esto da 4 combinaciones posibles que ilustran cómo cada elección afecta la velocidad y calidad del aprendizaje.

Variante	Algoritmo	Observación	Tiempo entreno	Dificultad
A	DQN	Simple 12D	Rápido	Baja
B	PPO	Simple 12D	Rápido	Baja
C	DQN	RGB imagen	Lento	Media
D	Comparativa	Simple 12D	—	—

8.4.1 Variante A — DQN + Observación Simple (*implementación actual*)

```
python flappybird_dqn.py --algorithm DQN --simple
```

Observación: vector de 12 valores (sensores LIDAR sintéticos) - Distancias a tuberías y suelo - Velocidad vertical del pájaro - Posición relativa

Algoritmo DQN (off-policy): - Guarda experiencias en un replay buffer (100K) - Aprende de muestras aleatorias del buffer → decorrelación - Más eficiente en datos: puede reutilizar experiencias viejas - Puede sobreestimar Q-values (problema conocido)

```
[13]: # Variante A: DQN + Observación Simple
# from flappybird_dqn import entrenar_flappy
# model, callback = entrenar_flappy(timesteps=100000, algorithm="DQN",
# ↪use_simple_obs=True)

print("Variante A: DQN + Observación Simple")
print("  Observación: 12D vector (LIDAR sintético)")
print("  Algoritmo: DQN (off-policy, replay buffer)")
print()
print("Configuración DQN:")
dqn_config = """
DQN(
    "MlpPolicy",          # Red densa (no CNN)
    env,
    learning_rate=0.0001,
    buffer_size=100000,   # Replay buffer: 100K experiencias
    learning_starts=10000,
    batch_size=32,
    gamma=0.99,
    exploration_fraction=0.1, # 10% del tiempo explorando
    exploration_final_eps=0.01,
)
"""
print(dqn_config)
```

Variante A: DQN + Observación Simple
Observación: 12D vector (LIDAR sintético)
Algoritmo: DQN (off-policy, replay buffer)

Configuración DQN:

```
DQN(
    "MlpPolicy",          # Red densa (no CNN)
    env,
    learning_rate=0.0001,
    buffer_size=100000,   # Replay buffer: 100K experiencias
    learning_starts=10000,
    batch_size=32,
    gamma=0.99,
    exploration_fraction=0.1, # 10% del tiempo explorando
    exploration_final_eps=0.01,
)
```

8.4.2 Variante B — PPO + Observación Simple

`python flappybird_dqn.py --algorithm PPO --simple`

Observación: misma que Variante A (12D vector)

Algoritmo PPO (on-policy): - No usa replay buffer: aprende solo de experiencias recientes - Actualiza la política con el *clipped surrogate objective* → estabilidad - Más robusto pero menos eficiente en datos (descarta experiencias pasadas) - Generalmente produce curvas de aprendizaje más suaves

Comparación A vs B con misma observación:

DQN: alta varianza, converge más rápido en datos, puede divergir

PPO: baja varianza, más pasos necesarios, más estable

```
[14]: # Variante B: PPO + Observación Simple
# model, callback = entrenar_flappy(timesteps=100000, algorithm="PPO",
# use_simple_obs=True)

print("Variante B: PPO + Observación Simple")
print(" Observación: 12D vector (igual que Var. A)")
print(" Algoritmo: PPO (on-policy, sin replay buffer)")
print()
print("Configuración PPO:")
ppo_config = """
PPO(
    "MlpPolicy",
    env,
    learning_rate=0.0003,
    n_steps=2048,      # Recoger 2048 pasos antes de actualizar
    batch_size=64,
    n_epochs=10,      # Reutilizar cada batch 10 veces
    gamma=0.99,
    clip_range=0.2,   # Clip del ratio política nueva/vieja
)
"""
print(ppo_config)
print("Diferencia clave: PPO reutiliza cada batch 10 veces (n_epochs=10)")
print("          DQN reutiliza miles de veces (del replay buffer)")
```

Variante B: PPO + Observación Simple

Observación: 12D vector (igual que Var. A)

Algoritmo: PPO (on-policy, sin replay buffer)

Configuración PPO:

```
PPO(
    "MlpPolicy",
    env,
```

```

    learning_rate=0.0003,
    n_steps=2048,      # Recoger 2048 pasos antes de actualizar
    batch_size=64,
    n_epochs=10,      # Reutilizar cada batch 10 veces
    gamma=0.99,
    clip_range=0.2,    # Clip del ratio política nueva/vieja
)

```

Diferencia clave: PPO reutiliza cada batch 10 veces (`n_epochs=10`)
DQN reutiliza miles de veces (del replay buffer)

8.4.3 Variante C — DQN + Imagen RGB

```
python flappybird_dqn.py --algorithm DQN
```

Observación: imagen completa del juego (288×512 píxeles RGB)

El agente ve los píxeles *exactamente igual que un humano* vería la pantalla. Esto requiere una CNN para extraer características visuales antes de decidir la acción.

Por qué es más difícil: 1. Espacio de observación enorme: 288×512×3 = 443K valores por frame
2. La CNN necesita aprender a detectar tuberías, movimiento, etc. 3. Requiere muchos más pasos de entrenamiento

Ventaja conceptual: no necesita ingeniería de características. El agente aprende qué mirar por sí solo.

Imagen 288×512×3 → CNN → features → MLP → acción
Vector 12D → MLP → acción (Var. A/B)

```

[15]: # Variante C: DQN + Imagen RGB
# model, callback = entrenar_flappy(timesteps=300000, algorithm="DQN",
# use_simple_obs=False)

print("Variante C: DQN + Imagen RGB")
print("  Observación: imagen 288×512×3 (píxeles)")
print("  Algoritmo: DQN con CnnPolicy")
print()
print("Diferencia en política:")
print("  Var. A/B: MlpPolicy → red densa (entrada: 12 valores)")
print("  Var. C:   CnnPolicy → CNN + red densa (entrada: imagen)")
print()
print("Configuración con imagen:")
cnn_config = """
env = gym.make("FlappyBird-v0", use_lidar=False) # Sin LIDAR → imagen completa

DQN(
    "CnnPolicy",    # CNN automática de Stable-Baselines3
    env,
    learning_rate=0.0001,

```

```

        buffer_size=100000,
        ...
    )
    """
    print(cnn_config)
    print("Nota: necesita ~3x más timesteps que la observación simple")

```

Variante C: DQN + Imagen RGB

Observación: imagen 288×512×3 (píxeles)

Algoritmo: DQN con CnnPolicy

Diferencia en política:

Var. A/B: MlpPolicy → red densa (entrada: 12 valores)

Var. C: CnnPolicy → CNN + red densa (entrada: imagen)

Configuración con imagen:

```
env = gym.make("FlappyBird-v0", use_lidar=False) # Sin LIDAR → imagen completa
```

```

DQN(
    "CnnPolicy", # CNN automática de Stable-Baselines3
    env,
    learning_rate=0.0001,
    buffer_size=100000,
    ...
)

```

Nota: necesita ~3x más timesteps que la observación simple

8.4.4 Variante D — Comparativa DQN vs PPO

```
python flappybird_dqn.py --compare-algorithms
```

0 para comparar las 4 combinaciones:

```
python flappybird_dqn.py --compare-all
```

Ejecuta múltiples variantes y genera gráficas comparativas para ver el impacto de cada elección de diseño.

```

[16]: # Variante D: Comparativa DQN vs PPO (misma observación simple)
      # from flappybird_dqn import comparar_algoritmos
      # resultados = comparar_algoritmos(timesteps=100000)

      print("Variante D: Comparativa DQN vs PPO")
      print("  Controla: misma observación simple (12D)")
      print("  Variable: algoritmo (DQN vs PPO)")
      print("  Genera: flappy_comparacion_algoritmos.png")
      print()
      print("Comparativa completa (4 combinaciones):")

```

```

print("  from flappybird_dqn import comparar_todo")
print("  resultados = comparar_todo(timesteps=80000)")
print("  Genera: flappy_comparacion_completa.png")
print()
print("Métricas de comparación:")
print("  - Score máximo alcanzado (tubos pasados)")
print("  - Media de score en los últimos 50 episodios")
print("  - Velocidad de convergencia (ep. para llegar a score > 5)")

```

Variante D: Comparativa DQN vs PPO

Controla: misma observación simple (12D)

Variable: algoritmo (DQN vs PPO)

Genera: flappy_comparacion_algoritmos.png

Comparativa completa (4 combinaciones):

from flappybird_dqn import comparar_todo

resultados = comparar_todo(timesteps=80000)

Genera: flappy_comparacion_completa.png

Métricas de comparación:

- Score máximo alcanzado (tubos pasados)
- Media de score en los últimos 50 episodios
- Velocidad de convergencia (ep. para llegar a score > 5)

8.4.5 Resumen: Cuándo Usar Cada Variante

Objetivo	Variante recomendada
Aprender rápido	A: DQN + Simple
Resultado más estable	B: PPO + Simple
Aprender solo de píxeles (sin ing. de features)	C: DQN + RGB
Comparar impacto del algoritmo	D: comparar-algorithms
Ver todo el panorama	D: compare-all

Lección principal: la elección de observación tiene más impacto que la elección de algoritmo. Un vector bien diseñado de 12 valores supera casi siempre a la imagen en entornos simples.